

Onyx — FileKit reference

Onyx · multi-process OS on Raspberry Pi 4

The reference of **FileKit** (`SD:/lib/filekit.so`, `user/Kits/filekit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: [The Kits](#).

Contents

1. What it is
2. Using it
3. Index
4. `filekit/filekit.h`
5. `filekit/fsutil.h`

What it is

FileKit is files and folders: whole files read and written, trees copied, moved and removed, paths, compression (zlib), and archives — ZIP read and written, tar / tar.gz / gzip read.

Include	<code>#include "filekit/filekit.h"</code>
Link	<code>lib/filekit.imp.a</code>
Library	<code>SD:/lib/filekit.so</code> — 78 entries in its table (<code>user/Kits/filekit/filekit.abi</code> , append-only)
Sources	<code>user/Kits/filekit/</code>

Using it

A whole file, read and written:

```
#include "filekit/filekit.h"
```

```
void *data; unsigned n;
if (fk_load ("SD:/docs/notes.txt", &data, &n) == 0)
{
    /* data: n bytes, a 0 after them */
    fk_save ("SD:/docs/notes.bak", data, n);
    fk_free (data);
}
```

Folders and paths:

```
#include "filekit/filekit.h"
```

```
char path[FS_PATHL];
fs_join (path, sizeof path, "SD:/docs", "report.txt");
if (!fs_exists (path)) { /* ... */ }
```

```
fk_mkdirs ("SD:/backup/2026/docs");           // every folder of the path
fk_copy ("SD:/docs", "SD:/backup/2026/docs", 0, 0); // a file, or a folder and all
it holds
fk_remove ("SD:/tmp/work");                     // the same, removed
```

A ZIP archive, read:

```
char err[80];
fk_zip *z = fk_zip_open ("SD:/downloads/pack.zip", err, sizeof err);
if (z)
{
    struct fk_zip_entry e;
    for (int i = 0; i < fk_zip_count (z); i++)
        if (fk_zip_entry (z, i, &e)) ax_putln (e.name);
    fk_zip_extract_all (z, 0, "SD:/downloads/pack", 0, 0);
    fk_zip_close (z);
}
```

A ZIP archive, written:

```
fk_zipw *w = fk_zipw_create ("SD:/backup/docs.zip");
fk_zipw_add (w, "SD:/docs", 0); // the folder and all it holds
fk_zipw_close (w, 0, 0, err, sizeof err);
```

Other archive formats (tar, tar.gz, gzip) are read through the same calls (`fk_arc_*`); a program asks the library which formats it handles (`fk_arc_formats`) instead of keeping its own list.

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
<code>fk_free</code>	a buffer the library returned	<code>filekit.h</code>
<code>fk_crc32</code>	(crc: 0 at first, then the last result)	<code>filekit.h</code>
<code>fk_adler32</code>	(adler: 1 at first)	<code>filekit.h</code>
<code>fk_deflate</code>	The whole of data compressed (level 0..9, -1)	<code>filekit.h</code>
<code>fk_inflate</code>	...	<code>filekit.h</code>
<code>fk_zip_entry</code>	(a type)	<code>filekit.h</code>
<code>fk_progress</code>	A step of a long work	<code>filekit.h</code>
<code>fk_zip_open</code>	0: err says why	<code>filekit.h</code>
<code>fk_zip_close</code>	the archive closed, its handle freed (0: nothing)	<code>filekit.h</code>
<code>fk_zip_error</code>	the last failure's words	<code>filekit.h</code>
<code>fk_zip_password</code>	for its encrypted entries	<code>filekit.h</code>
<code>fk_zip_count</code>	how many entries it holds (0: no archive)	<code>filekit.h</code>

Name	What it does	Header
<code>fk_zip_find</code>	its index (letters' case ignored), -1	<code>filekit.h</code>
<code>fk_zip_read</code>	entry i in memory (fk_free) → 0 / -1	<code>filekit.h</code>
<code>fk_zip_extract</code>	entry i as that file → 0 / -1	<code>filekit.h</code>
<code>fk_zip_extract_all</code>	Everything under prefix ("" or 0	<code>filekit.h</code>
<code>fk_zipw_create</code>	nothing written yet; 0: no path	<code>filekit.h</code>
<code>fk_zipw_add</code>	a file, or a folder and all it holds, as <code>name</code> (0: its own)	<code>filekit.h</code>
<code>fk_zipw_add_data</code>	a buffer as an entry (copied)	<code>filekit.h</code>
<code>fk_zipw_level</code>	0 store, 1 fast, 6 (the default), 9	<code>filekit.h</code>
<code>fk_zipw_close</code>	written → 0 / -1 (err); w freed	<code>filekit.h</code>
<code>fk_format</code>	The formats are asked, not assumed	<code>filekit.h</code>
<code>fk_arc_formats</code>	how many there are (out: up to max)	<code>filekit.h</code>
<code>fk_arc_probe</code>	What the file at <code>path</code> is	<code>filekit.h</code>
<code>fk_arc_is_name</code>	1: its extension is a format's that is read	<code>filekit.h</code>
<code>fk_arc_open</code>	any format read; 0: err says why	<code>filekit.h</code>
<code>fk_arc_new</code>	a new one, not on the card yet ("ZIP"; 0: not a format written)	<code>filekit.h</code>
<code>fk_arc_format</code>	"ZIP" ...	<code>filekit.h</code>
<code>fk_arc_path</code>	the archive file's path ("": no archive)	<code>filekit.h</code>
<code>fk_arc_comment</code>	the archive's own ("": none)	<code>filekit.h</code>
<code>fk_arc_writable</code>	1: it can be changed	<code>filekit.h</code>
<code>fk_arc_method_name</code>	entry i: "Deflate", "Store, crypted"...	<code>filekit.h</code>
<code>fk_arc_test</code>	entry i read and checked, nothing written → 0 / -1	<code>filekit.h</code>
<code>fk_extract</code>	(a type)	<code>filekit.h</code>
<code>fk_arc_extract_with</code>	0 / -1 (fk_zip_error)	<code>filekit.h</code>

Name	What it does	Header
<code>fk_arc_extract_bytes</code>	what that selection weighs	<code>filekit.h</code>
<code>fk_arc_add</code>	Changing an archive that can be (<code>fk_arc_writable</code>)	<code>filekit.h</code>
<code>fk_arc_add_conflicts</code>	names that exist already	<code>filekit.h</code>
<code>fk_arc_add_bytes</code>	the work's size	<code>filekit.h</code>
<code>fk_arc_delete</code>	Changing an archive that can be (<code>fk_arc_writable</code>)	<code>filekit.h</code>
<code>fk_arc_rename</code>	a file, or a folder and what is under it	<code>filekit.h</code>
<code>fk_arc_new_folder</code>	Changing an archive that can be (<code>fk_arc_writable</code>)	<code>filekit.h</code>
<code>fk_arc_write_empty</code>	a new archive put on the card with nothing in it	<code>filekit.h</code>
<code>fk_zipmem_count</code>	its entries, -1: not a ZIP	<code>filekit.h</code>
<code>fk_zipmem_entry</code>	Entry i	<code>filekit.h</code>
<code>fk_zipmem_get</code>	The entry of that name inflated (a 0 byte after its end) → 0 and *out (<code>fk_free</code>), -1	<code>filekit.h</code>
<code>fk_zipbuf_new</code>	an empty one; 0: no memory	<code>filekit.h</code>
<code>fk_zipbuf_add</code>	0 / -1	<code>filekit.h</code>
<code>fk_zipbuf_finish</code>	the archive (<code>fk_free</code>) → 0 / -1; b freed	<code>filekit.h</code>
<code>fk_zipbuf_free</code>	(given up before finish)	<code>filekit.h</code>
<code>fk_exists</code>	1 a file, 2 a folder, 0	<code>filekit.h</code>
<code>fk_file_size</code>	-1: no such file	<code>filekit.h</code>
<code>fk_load</code>	the whole file (a 0 byte after it; <code>fk_free</code>) → 0 / -1	<code>filekit.h</code>
<code>fk_save</code>	0 / -1	<code>filekit.h</code>
<code>fk_mkdirs</code>	every folder of the path made → 0 / -1	<code>filekit.h</code>
<code>fk_copy</code>	0 / -1	<code>filekit.h</code>
<code>fk_move</code>	renamed, else copied then removed	<code>filekit.h</code>
<code>fk_remove</code>	a file, or a folder and all it holds → 0 / -1	<code>filekit.h</code>
<code>fk_tree_size</code>	The bytes under a path (a file's size	<code>filekit.h</code>

Name	What it does	Header
<code>fk_path_name</code>	after the last '/' or ':' (inside path)	<code>filekit.h</code>
<code>fk_path_ext</code>	"png" (lower case, without the dot; "" : none)	<code>filekit.h</code>
<code>fk_path_folder</code>	"SD:/a/b.txt" → "SD:/a"; "SD:/a" → "SD:/"	<code>filekit.h</code>
<code>fk_path_join</code>	folder + "/" + name (no '/' added after a '/' or a ':', or before an empty name), cut to cap	<code>filekit.h</code>
<code>fk_path_unique</code>	a name not taken: "a.txt" → "a (2).txt"	<code>filekit.h</code>
<code>fk_human_size</code>	"12.4 KB"	<code>filekit.h</code>
<code>fk_dos_time_str</code>	"28/09/2026 14:12"	<code>filekit.h</code>
<code>fs_len</code>	a string's length in bytes (0 for a null pointer)	<code>fsutil.h</code>
<code>fs_copy</code>	s into d, cut to cap - 1 bytes, always ended by a 0	<code>fsutil.h</code>
<code>fs_lower</code>	an ASCII capital as its small letter, anything else unchanged	<code>fsutil.h</code>
<code>fs_ci_cmp</code>	strcmp with ASCII letters' case ignored: 0 equal, else < 0 or > 0	<code>fsutil.h</code>
<code>fs_join</code>	out = dir + "/" + name (no doubled slash).	<code>fsutil.h</code>
<code>fs_basename</code>	The last path component (points into path).	<code>fsutil.h</code>
<code>fs_dirname</code>	Parent directory of path into out ("SD:/a/b" → "SD:/a", "SD:/a" → "SD:/").	<code>fsutil.h</code>
<code>fs_is_dir</code>	Parent directory of path into out ("SD:/a/b" → "SD:/a", "SD:/a" → "SD:/").	<code>fsutil.h</code>
<code>fs_exists</code>	Parent directory of path into out ("SD:/a/b" → "SD:/a", "SD:/a" → "SD:/").	<code>fsutil.h</code>
<code>fs_copy_file</code>	Parent directory of path into out ("SD:/a/b" →	<code>fsutil.h</code>

Name	What it does	Header
	"SD:/a", "SD:/a" → "SD:/").	
<code>fs_skip_dot</code>	Parent directory of path into out ("SD:/a/b" → "SD:/a", "SD:/a" → "SD:/").	<code>fsutil.h</code>
<code>fs_copy_tree</code>	Copy a file or a whole folder tree (depth ≤ 12).	<code>fsutil.h</code>
<code>fs_remove_tree</code>	Delete a file or a whole folder tree (depth ≤ 12).	<code>fsutil.h</code>
<code>fs_unique_name</code>	A free path in dir for name	<code>fsutil.h</code>
<code>fs_text_fix</code>	A text file saved by a Windows editor	<code>fsutil.h</code>

filekit/filekit.h

filekit.h -- FileKit: what programs do with files, one copy for the whole system (SD:/lib/filekit.so; the shared libraries: docs/03 section 5.6). A program links lib/filekit.imp.a and calls plain C functions (fk_*):

```
compression  zlib's deflate and inflate (raw, zlib or gzip wrapped), CRC-32, Adler-32
archives      the formats known asked (fk_arc_formats), an archive of any of them
opened, extracted
              with choices, changed (add, delete, rename) -- ZIP read and written,
tar / tar.gz /
              gzip read
ZIP           an archive on the card read (its entries, one read to memory or
extracted, all of
              them) and made (files and folders of the card, buffers) -- the
Archiver's engine:
              ZIP64, old code pages, ZipCrypto passwords;
              an archive IN MEMORY found into and built (the office formats,
OpenRaster)
files         one loaded whole, saved; a file or a tree copied, moved, removed,
measured; folders made
paths         the name, the extension, the folder of a path; two parts joined; a
size for people
```

Everything is integer and pointers: a program built without the FPU calls it. A buffer the library returns (void **out) is freed with `fk_free`, never with `free` / `delete`. The first version: 2026-10-05; the second (the same day): archives of any format, asked from the library (`fk_arc_*`), tar and gzip read.

MIT License -- Copyright (c) 2026 Stéphane Wegener and the Onyx contributors. Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the

following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software. THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED.

```
void fk_free (void *p);           // a buffer the library returned
```

compression (zlib)

```
#define FK_RAW      0           // the deflate stream alone (a ZIP entry's)
#define FK_ZLIB     1           // with zlib's header and Adler-32 (PNG, PDF)
#define FK_GZIP     2           // with gzip's header and CRC-32 (.gz, HTTP)
#define FK_AUTO     3           // (inflate only) zlib or gzip, told from the first
bytes
```

```
unsigned fk_crc32 (unsigned crc, const void *data, unsigned n); // (crc: 0 at
first, then the last result)
unsigned fk_adler32 (unsigned adler, const void *data, unsigned n); // (adler: 1 at
first)
```

The whole of `data` compressed (level 0..9, -1: the default 6) → 0 and `*out` (`fk_free`) / `*out_n`, -1.

```
int fk_deflate (const void *data, unsigned n, int wrap, int level, void **out,
unsigned *out_n);
```

... and back. `size_hint`: the size expected (0: not known; the buffer grows) → 0 / -1 (damaged). The result has a 0 byte after its end (a text can be used as it is).

```
int fk_inflate (const void *data, unsigned n, int wrap, unsigned size_hint, void
**out, unsigned *out_n);
```

a ZIP archive on the card

```
struct fk_zip_entry
{
    char          name[300];       // its path in the archive (UTF-8, '/' between
the parts)
    unsigned long long size, packed; // the original size, what it takes in the
archive
    unsigned      crc, dos_time;   // CRC-32; the DOS date and time
(fk_dos_time_str)
    int           dir, encrypted;  // a folder; needs the password
    int           method;          // 0 stored, 8 deflate...
    int           reserved[4];
};
typedef struct fk_zip fk_zip;
```

A step of a long work: `done` of `total` bytes, the file being worked on (0: the same) → 0 to go on, anything else to stop.

```
typedef int (*fk_progress) (void *user, unsigned long long done, unsigned long long
total, const char *name);
```

```
fk_zip *fk_zip_open (const char *path, char *err, int cap); // 0: err says why
void fk_zip_close (fk_zip *z); // the archive closed, its handle freed (0: nothing)
const char *fk_zip_error (fk_zip *z); // the last failure's words
void fk_zip_password (fk_zip *z, const char *password); // for its encrypted
entries
int fk_zip_count (fk_zip *z); // how many entries it holds (0: no archive)
int fk_zip_entry (fk_zip *z, int i, struct fk_zip_entry *out); // 1 / 0
int fk_zip_find (fk_zip *z, const char *name); // its index (letters'
case ignored), -1
```

```
int fk_zip_read (fk_zip *z, int i, void **out, unsigned *out_n);    // entry i in
memory (fk_free) -> 0 / -1
int fk_zip_extract (fk_zip *z, int i, const char *dest_path);    // entry i as
that file -> 0 / -1
```

Everything under prefix ("" or 0: the whole archive) into the folder dest_dir, the folders made, the names made safe for the card → the files written, -1 (fk_zip_error; stopped: what was done stays).

```
int fk_zip_extract_all (fk_zip *z, const char *prefix, const char *dest_dir,
fk_progress cb, void *user);
```

A new archive (an existing file is replaced when it is closed): its entries said first, written by fk_zipw_close.

```
typedef struct fk_zipw fk_zipw;
fk_zipw *fk_zipw_create (const char *path); // nothing written yet; 0: no path
int fk_zipw_add (fk_zipw *w, const char *disk_path, const char *name); // a file, or
a folder and all it holds, as `name` (0: its own)
int fk_zipw_add_data (fk_zipw *w, const char *name, const void *data, unsigned n);
// a buffer as an entry (copied)
void fk_zipw_level (fk_zipw *w, int level);    // 0 store, 1 fast, 6 (the
default), 9
int fk_zipw_close (fk_zipw *w, fk_progress cb, void *user, char *err, int cap); //
written -> 0 / -1 (err); w freed
```

archives of any format the library knows (version 2, 2026-10-05)

The formats are asked, not assumed: a program that shows archives (the Archiver) lists them from here, so a format added to the library is one it has at once.

```
struct fk_format
{
    char name[16];    // "ZIP", "TAR", "TAR.GZ", "GZIP"
    char extensions[96];    // "zip jar docx ..." (lower case, a space
between)
    char note[80];    // what can be done with it, in a few words
    int can_read, can_write;    // opened and extracted; made and changed (add,
delete, rename)
    int can_password;    // encrypted entries are read (with fk_arc_password)
    int reserved[4];
};
int fk_arc_formats (struct fk_format *out, int max);    // how many there are
(out: up to max)
```

What the file at path is: 1 and its format's name -- or 0 and why ("7z archives are not supported yet.").

```
int fk_arc_probe (const char *path, char *format, int fcap, char *why, int wcap);
int fk_arc_is_name (const char *name);    // 1: its extension is a
format's that is read
```

An archive opened: the same handle as fk_zip (fk_zip_count, _entry, _find, _read, _extract, _extract_all, _password, _error and _close work on it whatever its format).

```
typedef struct fk_zip fk_arc;
fk_arc *fk_arc_open (const char *path, char *err, int cap);    // any format read;
0: err says why
fk_arc *fk_arc_new (const char *path, const char *format);    // a new one, not on
the card yet ("ZIP"; 0: not a format written)
const char *fk_arc_format (fk_arc *a);    // "ZIP" ...
const char *fk_arc_path (fk_arc *a);    // the archive file's path (": no archive)
```

```

const char *fk_arc_comment (fk_arc *a);           // the archive's own ("":
none)
int fk_arc_writable (fk_arc *a);                  // 1: it can be changed
int fk_arc_method_name (fk_arc *a, int i, char *out, int cap); // entry i:
"Deflate", "Store, crypted"...
int fk_arc_test (fk_arc *a, int i, fk_progress cb, void *user); // entry i read and
checked, nothing written -> 0 / -1

```

Extracting with choices.

```

#define FK_LAYOUT_FULL      0      // the archive's folders kept
#define FK_LAYOUT_FROM      1      // ... from the folder `from` down (what is above
it dropped)
#define FK_LAYOUT_FLAT      2      // the files alone, no folder
#define FK_EXISTS_ASK       0      // a file that exists: ask (the callback),
replace, skip, keep both
#define FK_EXISTS_REPLACE   1
#define FK_EXISTS_SKIP      2
#define FK_EXISTS_KEEP_BOTH 3
#define FK_ANSWER_REPLACE   1      // the callback's answers
#define FK_ANSWER_SKIP      2
#define FK_ANSWER_KEEP_BOTH 3
#define FK_ANSWER_REPLACE_ALL 4
#define FK_ANSWER_SKIP_ALL  5
#define FK_ANSWER_CANCEL    (-1)
struct fk_extract
{
    unsigned    size;           // sizeof (struct fk_extract)
    const char  *dest;          // the folder to extract into (made)
    const char  *selected;      // one byte an entry, not 0: taken (0: every
entry)
    const char  *from;          // FK_LAYOUT_FROM: the archive's folder shown
    int         layout, exists;
    int         (*ask) (void *user, const char *path); // FK_EXISTS_ASK -> an
FK_ANSWER_*
    fk_progress progress;
    void        *user;          // (both callbacks')
    int         files, skipped;  // out: what was done
};
int fk_arc_extract_with (fk_arc *a, struct fk_extract *x); // 0 / -1
(fk_zip_error)
unsigned long long fk_arc_extract_bytes (fk_arc *a, const char *selected); // what
that selection weighs

```

Changing an archive that can be (fk_arc_writable): it is rewritten beside itself, then swapped in, and its entries are read again (the indexes change). → 0 / -1 (fk_zip_error). paths: files and folders of the card, put in the archive's folder into ("": its top); keep_folders: a folder keeps its name and tree; replace: a name that exists is replaced, else skipped.

```

int fk_arc_add (fk_arc *a, const char *const *paths, int n, const char *into, int
keep_folders, int replace, int level,
    fk_progress cb, void *user, int *added);
int fk_arc_add_conflicts (fk_arc *a, const char *const *paths, int n, const char
*into, int keep_folders); // names that exist already
unsigned long long fk_arc_add_bytes (fk_arc *a, const char *const *paths, int n,
const char *into, int keep_folders, int replace); // the work's size
int fk_arc_delete (fk_arc *a, const char *selected, fk_progress cb, void *user);
int fk_arc_rename (fk_arc *a, const char *from, const char *to, fk_progress cb, void
*user); // a file, or a folder and what is under it
int fk_arc_new_folder (fk_arc *a, const char *name, fk_progress cb, void *user);

```

```
int fk_arc_write_empty (fk_arc *a); // a new archive put on the card
with nothing in it
```

a ZIP archive in memory (a document read whole: .docx, .xlsx, .odt, .ora ...)

```
int fk_zipmem_count (const void *zip, unsigned n); // its entries, -1: not a
ZIP
```

Entry i: its name (cap bytes) and its size → 1 / 0.

```
int fk_zipmem_entry (const void *zip, unsigned n, int i, char *name, int cap,
unsigned *size);
```

The entry of that name inflated (a 0 byte after its end) → 0 and *out (fk_free), -1: none, damaged.

```
int fk_zipmem_get (const void *zip, unsigned n, const char *name, void **out,
unsigned *out_n);
```

... and one built: entries added (level 0: stored -- the "mimetype" of an OpenDocument first), then the archive's bytes.

```
typedef struct fk_zipbuf fk_zipbuf;
fk_zipbuf *fk_zipbuf_new (void); // an empty one; 0: no memory
int fk_zipbuf_add (fk_zipbuf *b, const char *name, const void *data, unsigned n, int
level); // 0 / -1
int fk_zipbuf_finish (fk_zipbuf *b, void **out, unsigned *out_n); // the archive
(fk_free) -> 0 / -1; b freed
void fk_zipbuf_free (fk_zipbuf *b); // (given up before finish)
```

files

```
int fk_exists (const char *path); // 1 a file, 2 a folder, 0
long long fk_file_size (const char *path); // -1: no such file
int fk_load (const char *path, void **out, unsigned *out_n); // the whole file
(a 0 byte after it; fk_free) -> 0 / -1
int fk_save (const char *path, const void *data, unsigned n); // 0 / -1
int fk_mkdirs (const char *path); // every folder of the path made
-> 0 / -1
```

A file, or a folder and all it holds. `dst` is the new path itself (not the folder to put it in).

```
int fk_copy (const char *src, const char *dst, fk_progress cb, void *user); // 0 / -1
int fk_move (const char *src, const char *dst, fk_progress cb, void *user); //
renamed, else copied then removed
int fk_remove (const char *path); // a file, or a folder and all it
holds -> 0 / -1
```

The bytes under a path (a file's size; a folder's whole tree); *files, *folders: how many (0: not wanted).

```
long long fk_tree_size (const char *path, int *files, int *folders);
```

paths

```
const char *fk_path_name (const char *path); // after the last '/' or
':' (inside path)
void fk_path_ext (const char *path, char *out, int cap); // "png" (lower case,
without the dot; "": none)
void fk_path_folder (const char *path, char *out, int cap); // "SD:/a/b.txt" ->
"SD:/a"; "SD:/a" -> "SD:/"
void fk_path_join (char *out, int cap, const char *folder, const char *name); //
folder + "/" + name (no '/' added after a '/' or a ':', or before an empty name), cut
```

```

to cap
void fk_path_unique (char *path, int cap);           // a name not taken: "a.txt"
-> "a (2).txt"
void fk_human_size (unsigned long long bytes, char *out, int cap); // "12.4 KB"
void fk_dos_time_str (unsigned dos_time, char *out, int cap);      // "28/09/2026
14:12"
}
#endif

```

(the file helpers -- fs_join, fs_exists, fs_copy_tree...: fsutil.h, C++)

filekit/fsutil.h

fsutil.h -- file-system helpers shared by the file tools (File Viewer, trash, dock): path join, exists / is_dir, recursive copy and delete, a free "name copy" name. FileKit's (filekit.so): this header declares, fsutil.inc has the code (it was user/fsutil.h, a header of inline functions, until 2026-10-05). C++.

(a program for Onyx: FileKit's functions, by name. A PC build, or FS_INLINE: the code inline in the program -- no shared library there.)

```

#define FS_API extern "C"
#define FS_API static inline

#define FS_NAMELEN 72
#define FS_PATHLEN 300
int fs_len (const char *s); // a string's length in bytes (0 for a null pointer)
void fs_copy (char *d, const char *s, int cap); // s into d, cut to cap - 1 bytes,
always ended by a 0
char fs_lower (char c); // an ASCII capital as its small letter, anything else
unchanged
int fs_ci_cmp (const char *a, const char *b); // strcmp with ASCII letters' case
ignored: 0 equal, else < 0 or > 0

```

out = dir + "/" + name (no doubled slash).

```
void fs_join (char *out, int cap, const char *dir, const char *name);
```

The last path component (points into path).

```
const char *fs_basename (const char *path);
```

Parent directory of path into out ("SD:/a/b" → "SD:/a", "SD:/a" → "SD:/").

```

void fs_dirname (char *out, int cap, const char *path);
bool fs_is_dir (const char *path);
bool fs_exists (const char *path);
bool fs_copy_file (const char *src, const char *dst);
bool fs_skip_dot (const char *n);

```

Copy a file or a whole folder tree (depth ≤ 12).

```
bool fs_copy_tree (const char *src, const char *dst, int depth = 0);
```

Delete a file or a whole folder tree (depth ≤ 12).

```
bool fs_remove_tree (const char *path, int depth = 0);
```

A free path in dir for name: "name", then "name copy", "name copy 2", ... (the extension stays at the end: "notes copy.txt").

```
void fs_unique_name (char *out, int cap, const char *dir, const char *name, const char *tag = " copy");
```

A text file saved by a Windows editor: drop a UTF-8 BOM, turn UTF-16 (Notepad) into 8-bit. In place (b[n] must be writable); returns the new length.

```
int fs_text_fix (char *b, int n);
```